

Diagrama de Pacotes do RadarTorres

Protótipo para detecção, localização e seleção automática de ação por sensoriamento — TCC, Engenharia de Computação, Faculdade UCL

1. O que é um diagrama de pacotes

O diagrama de pacotes é um diagrama **estrutural** da UML que representa a organização lógica de um sistema em **pacotes** — agrupadores de elementos de modelagem, como classes, interfaces e outros pacotes — e as **dependências** existentes entre eles. Diferentemente do diagrama de classes, ele não detalha atributos nem métodos: situa-se num nível de abstração superior e responde à pergunta *como o sistema está dividido e quem depende de quem*.

Na notação UML, o pacote é desenhado como um retângulo com uma aba no canto superior esquerdo, semelhante a uma pasta, podendo conter outros pacotes aninhados. A dependência é uma seta tracejada com ponta aberta, orientada do pacote que *utiliza* para o pacote *utilizado*. Seu principal uso é evidenciar a arquitetura em camadas, verificar se as responsabilidades estão bem separadas e identificar acoplamentos indesejados, como dependências cíclicas.

2. Aplicação ao RadarTorres

No RadarTorres, aplicação desktop em C#/WPF (.NET 9) que segue o padrão MVVM, cada pacote corresponde a um **namespace** do projeto RadarTorres.App, que por sua vez coincide com uma pasta do código-fonte. O diagrama foi obtido diretamente do código da branch principal do repositório: cada seta representa ao menos uma diretiva `using RadarTorres.App.*` (ou uma referência `clr-namespace` em XAML) presente nos arquivos do pacote de origem. Dessa forma, o diagrama descreve a estrutura efetivamente implementada, e não apenas a pretendida.

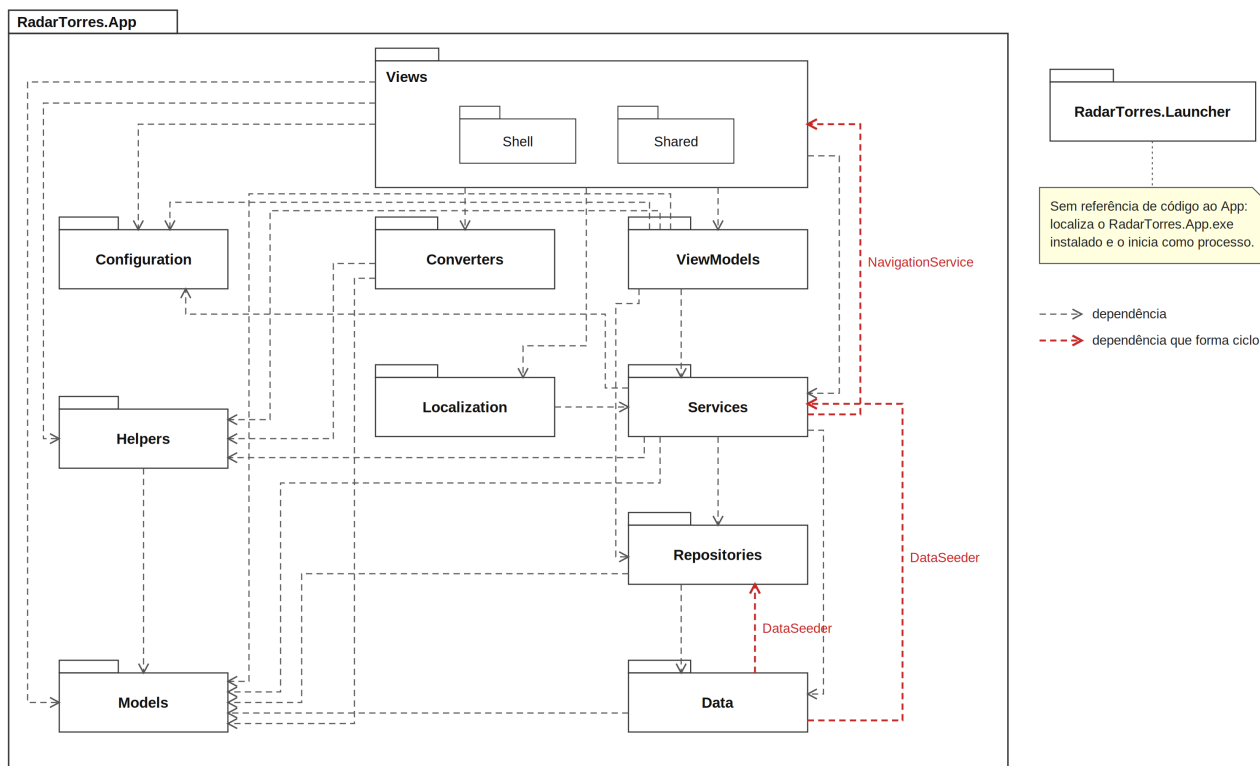


Figura 1. Diagrama de pacotes do RadarTorres. Setas em vermelho indicam dependências que formam ciclos (dos autores).

3. Responsabilidade de cada pacote

Pacote	Responsabilidade	Exemplos de elementos
Views (Shell, Shared)	Interface gráfica em XAML. Shell contém a janela principal e as barras de navegação; Shared reúne controles reutilizáveis.	MonitoramentoView, RadarControl, ShellWindow, DashboardCanvas
Converters	Conversores de binding do WPF, que traduzem estados do modelo em cores, rótulos e visibilidade.	ConnectionStateToBrushConverter, QuadrantToLabelConverter
Localization	Extensão de marcação XAML para exibir textos traduzidos (pt-BR/en-US).	LocExtension
ViewModels	Estado e comandos das telas; orquestram os serviços sem conter regra de negócio.	MainViewModel, ShellViewModel, LoginViewModel
Services	Regras de negócio: comunicação serial, rastreamento, seleção de torre, acionamento demonstrativo, simulação, autenticação, permissões, tema, idioma, câmeras e Arduino CLI.	SerialCommunicationService, TowerSelectionService, SimulationService, AuthService
Repositories	Contratos de acesso a dados (interfaces) e suas implementações atuais em CSV.	IUsuarioRepository, CsvUsuarioRepository
Data	Infraestrutura de persistência em CSV: caminhos, leitura/escrita de tabelas e carga inicial.	CsvTableStore, AppDataPaths, DataSeeder
Configuration	Representação das configurações lidas do appsettings.json.	AppSettings, ArduinoCliSettings
Helpers	Utilitários de cálculo e de apoio ao MVVM.	CoordinateConverter, DistanceCalculator, RelayCommand
Models	Entidades do domínio. Não depende de nenhum outro pacote.	Target, Tower, SensorReading, Usuario
RadarTorres. Launcher	Executável auxiliar que localiza a aplicação instalada e a inicia como processo, sem referência de código a ela.	Program

4. Leitura das dependências

O fluxo principal segue a organização do MVVM, de cima para baixo: **Views** → **ViewModels** → **Services** → **Repositories** → **Data** → **Models**. As Views também usam Converters e Localization para apresentação, e Models, Helpers e Configuration funcionam como pacotes de base, consumidos pelas camadas superiores. O pacote Models ocupa a posição mais estável do sistema: todos dependem dele e ele não depende de ninguém, o que é desejável para as entidades do domínio. A separação entre as interfaces de Repositories e a implementação em CSV de Data é o que viabiliza a futura migração para um banco relacional sem alterar ViewModels e Services.

5. Pontos de atenção identificados

A construção do diagrama a partir do código revelou três dependências que formam ciclos entre pacotes, destacadas em vermelho na Figura 1:

- **Services** → **Views**: o `NavigationService` importa os namespaces de Views para instanciar as telas. Com isso, a camada de serviços passa a conhecer a camada de interface, contrariando a separação pretendida pelo MVVM.
- **Data** → **Repositories** e **Data** → **Services**: o `DataSeeder`, responsável pela carga inicial de dados, utiliza repositórios e serviços, enquanto esses mesmos pacotes dependem de Data.

Esses ciclos não impedem o funcionamento da aplicação, pois todos os pacotes pertencem ao mesmo assembly, mas aumentam o acoplamento e dificultam testes isolados e futuras separações em projetos distintos. Registrá-los faz parte do propósito do diagrama: tornar visíveis decisões de arquitetura que, apenas pelo código, passariam despercebidas.

6. Escopo e observações

As dependências do namespace raiz (`App.xaml.cs`) foram omitidas, pois ele atua como *composition root* da injeção de dependência e referencia quase todos os pacotes, o que prejudicaria a legibilidade. A pasta `ComputerVision` não foi representada por conter apenas o modelo de detecção (arquivo `ONNX`), sem código C#. O projeto de testes automatizados não está presente na branch analisada e, por isso, também não foi incluído.